Abhishek Smitha Biju

BITS Pilani, Goa Campus · B.E. Computer Science, 2024 – 2028 · CGPA 7.42 / 10
+91-9747801887 · f20240866@goa.bits-pilani.ac.in · github.com/abhishek-sbiju · abhishek-sbiju.github.io

Experience

Software Engineering Intern — FourKites **May 2026 – Jul 2026**
Test Automation / SDET · Dynamic Yard Management System *Chennai, India*

- Authored and published the **approved HLD** for the VirtuosoQA → Playwright test migration; a senior engineer then built the team's base migration tooling **per that HLD**. Rewrote v2 after parsing six real exports, because v1 was a paper design.
- Built a **deterministic converter** — same output every run, so a migration could be diffed against production instead of trusted. Validated on 291- and 527-step exports: zero raw XPath, zero unknown actions, low-confidence selectors flagged rather than silently emitted.
- Migrated **23 recorded journeys** into production Playwright-BDD across **20 merged PRs**, each healed green against a live QA environment. Filed a genuine application defect rather than masking a filter modal that rendered no options.
- Built a **dual-engine validation lane**: every migrated test is reconciled against its original recording by an oracle that **hard-blocks false-greens**, behind four zero-cost build-time gates that run before the ~10-minute QA run.
- Moved the mechanical 250-step parse out of the LLM into a Node converter — my proposal — cutting per-journey turnaround from **~4 h to ~90 min** (self-measured) and making runs reproducible.

Founder — Versor **Jun 2026 – Present**
versor.in — smart menu platform for restaurants

- One menu, every surface a guest can see. A restaurant edits in Google Sheets or a dashboard; its QR menu, website and SEO pages all publish from that one source, so nothing goes stale in one place and not another. **Built and deployed for 13 restaurants and cafés.**
- Sat with **70+ restaurant owners** before building most of it and **invoiced three venues at ₹9–10k each**, turning near-identical client builds into a templated product. Live: ak7royalemirates.com, madras-square.vercel.app, ccc-menu.vercel.app — staff change dishes, themes and photos themselves, without calling me.
- Extracted **menu-core**, a typed sync engine (Sheet → canonical menu model, **385 lines of tests**), after **13 forked copies** of the per-client parser drifted apart. Chose Google's CSV export over its typed JSON endpoint, which **silently nulls** a price typed `9000 (1000ml)` — a silent null is worse than a parse error.
- Built and run solo end to end — Next.js, the Sheets sync, Firebase auth and storage, deployment — including production support when a live menu breaks during service.

Projects

Invariant — Learn · algorithms and chess trainer **Jul 2026 – Present**
invariant.org.in/learn/dsa

- Set one rule and built the machinery to enforce it: **nothing reaches a reader that the build has not checked.** Lessons are generated from data by a Python pipeline, then audited for design-token drift and drill-answer bias — and the audit reads the **built** HTML, so a page that forgot to rebuild fails too.
- **1,128 chess exercises decoded from the book's own diagram glyphs** rather than typed in: the font encodes each square's colour in letter case, so a mis-sliced grid contradicts itself on the first row. Every solution line is then replayed for legality. **Stockfish runs but is deliberately not allowed to gate anything** — several puzzles have more than one winning move, so grading the authors against an engine would replace their judgement with a number.
- Quantitative problems must be **independently recomputed** before they may appear, and every numeral a check relies on is audited against the question text, otherwise a misread question has the checker correctly solve the wrong problem. The gate is itself tested by corrupting a number and proving the build fails.

Invariant — Life · offline-first personal operations system **Jul 2026 – Present**
invariant.org.in — owner-only; walkthrough available

- Work, intake, training, timetable, money and a journal in one dashboard. An Android app writes to Firestore and the laptop reflects it the same second; both halves work offline and reconcile afterwards.
- **Two halves, two opposite sync rules, by design.** Study progress merges so it cannot be lost — counts take the max, review dates the earliest. The life log is last-write-wins per field on purpose, because correcting a weight from 68.4 to 66.4 must not be discarded by a `max()`; it is made safe by a schema rule that nothing two devices can touch is ever an array.
- One of **63 test suites** asserts the **deployed** security rules match the repository — it performs reads that correct rules must refuse, and `permission-denied` is the pass. Writing it found a real hole: Firestore evaluates rules with OR, so a blanket grant followed by a deny never revoked anything.
- *Architecture, schema and correctness rules mine; application code largely AI-assisted.*

Technical Skills

Languages: C++, Python, JavaScript, TypeScript, SQL · **Web:** React.js, Next.js, Node.js, Firebase / Firestore
Testing: Playwright, playwright-bdd (Gherkin), test-data & selector strategy · **Tools:** Git, Vercel · **Core CS:** DSA, OOP, DBMS

Competitive Programming

- **Codeforces Specialist — 1512** · **LeetCode contest rating 1598** (top 24% globally) · codeforces.com/profile/Abhishek_S_Biju